Prerrequisitos y notación

Genesis Silesky

20 de octubre de 2026

1. Prerrequisitos y notación

Este capítulo reúne las herramientas matemáticas y de software que usan todos los demás. Cada capítulo de concepto indica, en sus cajas azules, qué secciones de aquí necesita; si algo no se entiende, conviene volver a la sección correspondiente antes de continuar.

1.1. Números complejos

Un número complejo es $z = a + bi$ con $a, b \in \mathbb{R}$ e $i^2 = -1$.

- **Conjugado:** $\bar{z} = a - bi$.
- **Módulo:** $|z| = \sqrt{z\bar{z}} = \sqrt{a^2 + b^2}$. Es el que importa para las probabilidades.
- **Forma polar y fórmula de Euler:** $z = |z|e^{i\varphi}$, con $e^{i\varphi} = \cos\varphi + i\sin\varphi$. El ángulo φ es la *fase*.

En Python un complejo se escribe `1+1j` (la unidad imaginaria es `1j`) y `np.abs(z)` devuelve su módulo.

1.2. Vectores, norma y producto interno

Los estados de n qubits viven en $\mathbb{C}^{2^n}$. Para vectores columna $u, v \in \mathbb{C}^d$ el producto interno es $\langle u|v\rangle = \sum_k \bar{u}_k v_k$ y la norma es $\|v\| = \sqrt{\langle v|v\rangle}$. Un estado cuántico válido siempre tiene $\|v\| = 1$ ("normalizado").

1.3. Notación de Dirac

Símbolo	Significado	En Python/numpy
$\|\psi\rangle$	vector columna ("ket")	`np.array([a, b])`
$\langle\psi\|$	su transpuesta conjugada ("bra")	`v.conj()`
$\langle\varphi\|\psi\rangle$	producto interno	`np.vdot(phi, psi)`
$\|0\rangle = \binom{1}{0},\ \|1\rangle = \binom{0}{1}$	base computacional	`Statevector.from_label("0")`

1.4. Matrices unitarias

La adjunta de U es $U^\dagger = \bar{U}^T$ (transpuesta conjugada). U es *unitaria* si $U^\dagger U = UU^\dagger = I$. Las matrices unitarias conservan la norma: $\|Uv\| = \|v\|$, y son invertibles con $U^{-1} = U^\dagger$. Toda puerta cuántica es una matriz unitaria.

1.5. Producto tensorial (Kronecker)

Para combinar sistemas se usa $A \otimes B$. En vectores, $|a\rangle \otimes |b\rangle$ se escribe $|ab\rangle$; por ejemplo $|0\rangle \otimes |1\rangle = |01\rangle = (0, 1, 0, 0)^T$. Con n qubits la base tiene 2^n vectores $|x\rangle$, $x \in \{0, 1\}^n$. Un estado que *no* puede escribirse como $|a\rangle \otimes |b\rangle$ está entrelazado (capítulo 05b).

1.6. Probabilidad básica

- **Regla de Born:** si el estado es $\sum_x c_x |x\rangle$, medir da x con probabilidad $|c_x|^2$ (y esas probabilidades suman 1 por la normalización).
- **Frecuencia y ley de los grandes números:** al repetir N veces un experimento con probabilidad p, la frecuencia observada tiene error típico $\sqrt{p(1-p)/N}$ y tiende a p.

1.7. Conceptos de Qiskit

Circuito (`QuantumCircuit`) Lista ordenada de puertas y mediciones sobre qubits y bits clásicos.

`Statevector` Calcula el vector de estado *exacto* de un circuito sin medir; solo es posible en simulación.

Simulador (`AerSimulator`) Ejecuta circuitos en el computador local, con o sin ruido.

Shot Una ejecución completa del circuito con su medición. Se repite muchas veces para estimar probabilidades.

Conteos (`get_counts`) Diccionario cadena-de-bits → número de veces que salió.

Orden de los bits en Qiskit (little-endian)

Qiskit escribe las cadenas con el qubit 0 a la **derecha**: la cadena `'10'` significa $q_1 = 1$, $q_0 = 0$. En estos ejemplos se señala cada vez que importa.

1.8. Puertas usadas en estos ejemplos

Puerta	Matriz	Efecto	Método Qiskit
H	$\frac{1}{\sqrt{2}}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix}$	superposición	`qc.h(q)`
X	$\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix}$	NOT: $\|0\rangle \leftrightarrow \|1\rangle$	`qc.x(q)`
Z	$\begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix}$	cambia el signo de $\|1\rangle$	`qc.z(q)`
$R_y(\theta)$	$\begin{pmatrix}\cos\frac{\theta}{2} & -\sin\frac{\theta}{2}\\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2}\end{pmatrix}$	rotación real	`qc.ry(theta, q)`
CNOT	(4×4)	invierte el objetivo si el control es $\|1\rangle$	`qc.cx(c, t)`

1.9. Cómo ejecutar el código

Cada ejemplo existe como `.py` (terminal) y como `.ipynb` (Jupyter/Colab). En el `.py`, las líneas `#%%` separan "celdas": en un notebook cada una es una celda; en la terminal no tienen efecto. Instalación (versiones con las que se probó todo):

```
pip install qiskit==2.5.0 qiskit-aer==0.17.2 matplotlib pylatexenc
python 01_qubit_vector.py
```

La API de Qiskit ha cambiado bastante entre versiones; si se usa otra versión algún ejemplo podría fallar.